

1.3 Lesson: Regression and Classification with Neural Networks



Regression and Classification with Neural Networks

Regression with Neural Networks

For **regression** problems, the non-linear activation function is omitted in the single output neuron, which produces a floating-point number.

The **loss (or cost or error) function** remains exactly the same as with non-deep regression algorithms: Mean Squared Error (MSE), Root Mean Squared Error (RMSE), etc.

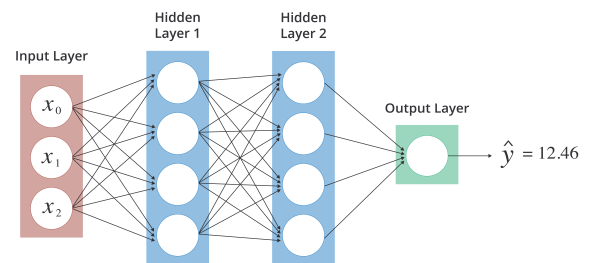


Figure 1: *Regression Network*

Classification with Neural Networks

Recall from Module 3 that we are interested in three different types of classification problems, defined by how many classes exist for our task and how many we wish to identify:

Binary
Classification

- Spam
- Not spam

Multiclass
Classification

- Dog
- Cat
- Horse
- Fish
- Bird
- ...

Multi-label
Classification

- Dog
- Cat
- Horse
- Fish
- Bird
- ...

Figure 2: Three Types of Classification Problems

Source: Paul, Saugata. (2019, June 16). A detailed case study on multi-label classification with Machine Learning algorithms and predicting movie tags based on plot summaries!
(<https://medium.com/@saugata.paul1010/a-detailed-case-study-on-multi-label-classification-with-machine-learning-algorithms-and-72031742c9aa>) *Medium*.

Binary Classification

For **binary classification** tasks, there is one class, and we wish to predict whether the sample is in the class or not; the output layer is a logistic regression classifier with the sigmoid activation function, producing a probability $\hat{y} \in [0, 1]$ and applying the decision rule that the input is **in** the class if $\hat{y} \geq 0.5$ and **not in** the class if $\hat{y} < 0.5$:

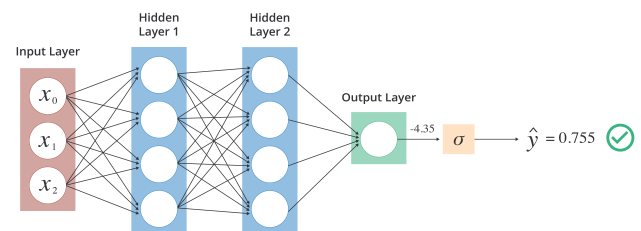
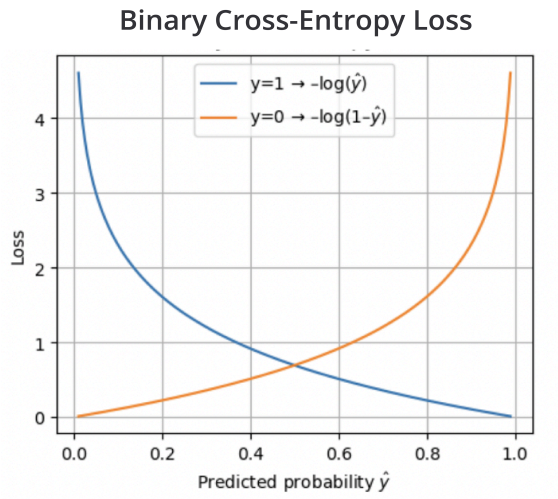


Figure 3: Binary Classification Network

We use **binary cross-entropy** as the loss function, which is designed to give increasingly large penalties to wrong predictions for the actual value y :

$$\text{loss}(\hat{y}) = \begin{cases} -\log(\hat{y}) & \text{if } y = 1 \\ -\log(1 - \hat{y}) & \text{if } y = 0 \end{cases}$$



Derivation

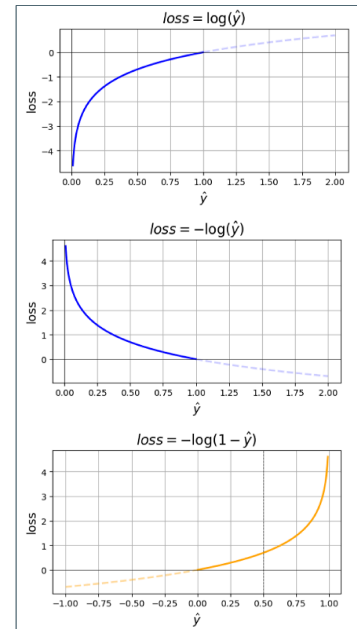


Figure 4: Binary Cross Entropy Loss

This means that:

- approximately correct predictions (e.g., $\hat{y} \approx y$) will have a small loss, and
- extremely wrong predictions (e.g., $1 - \hat{y} \approx y$) will have a huge loss.

The standard (and somewhat obscure) formula for binary cross-entropy loss defines the loss L over all input samples as the **average** over all n samples:

Sure! Here's the LaTeX code for the given equation:

$$L = -\frac{1}{n} \sum_{i=1}^n (y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}))$$

where:

$y^{(i)} \in \{0,1\}$ = ground truth for sample i ,

$\hat{y}^{(i)} \in [0,1]$ = predicted probability for sample i .

(Notice how the negative sign is pulled to the very front of the formula and that $y^{(i)}$ will “zero-out” one side or the other of the sum, selecting the appropriate log formula.)

Multi-Label Classification

For multi-label classification tasks, there is more than one class, and we effectively wish to perform binary classification simultaneously on all classes:

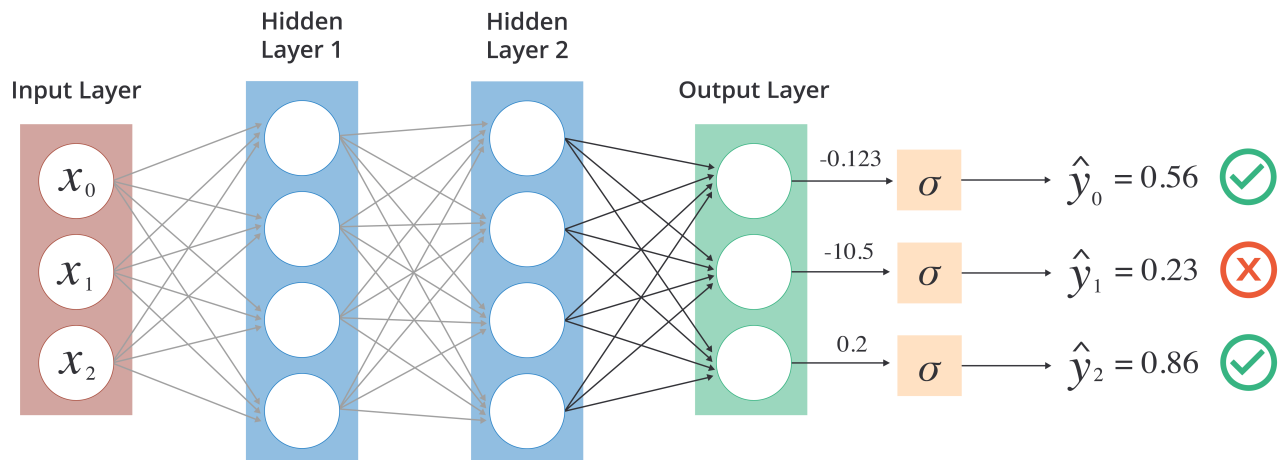


Figure 5: Multi-Label Classification Network

Ground truth in this case is a “one-hot” encoded vector $[y_1, \dots, y_C]$ of 0's and 1's, where C = the number of classes. The **cross-entropy** loss function in this case is simply the average of the binary cross entropy over all samples and all classes:

$$L = -\frac{1}{nC} \sum_{i=1}^n \sum_{j=1}^C (y_j^{(i)} \log \hat{y}_j^{(i)} + (1 - y_j^{(i)}) \log(1 - \hat{y}_j^{(i)}))$$

where:

- $y_j^{(i)} \in \{0, 1\}$ = ground truth for label j of sample i ,
- $\hat{y}_j^{(i)} \in [0, 1]$ = predicted probability for label j of sample i .

Next week, we'll consider how to implement **multi-class classification** using the **softmax** function and understand how **gradient descent** and **optimization algorithms** may be used in conjunction with the loss function to adjust the weights in a neural network to solve regression and classification problems.

Conclusion

Neural networks form the foundation of deep learning. By stacking layers of neurons and optimizing their weights through gradient-based learning, they can model complex functions and patterns. Understanding the basic structure — neurons, activation functions, and network layers — is key to leveraging deep learning for practical applications.